CV-Week3-Notes

CNN Architectures

1. Introduction
   - CNNs (Convolutional Neural Networks) are the backbone of modern computer vision.
   - They evolved over time to solve increasingly complex image tasks.
   - Studying landmark architectures helps students understand **how design choices improved performance** and shaped today's models.

2. LeNet (1998)
   - **Inventor:** Yann LeCun.
   - **Purpose:** Digit recognition (MNIST dataset).
   - **Architecture:** Convolution → Pooling → Fully Connected layers.
   - Key Contribution:
     - First proof that CNNs outperform traditional ML methods (like SVMs) for images.
     - Inspired early adoption in handwriting recognition and banking systems.

3. AlexNet (2012)
   - **Event:** Won the ImageNet competition by a huge margin.
   - **Architecture:** 5 convolutional layers + 3 fully connected layers.
   - Innovations:
     - ReLU activation (faster training).
     - Dropout (reduces overfitting).
     - GPU training (massive speed-up).
   - **Key Contribution:** Sparked the deep learning revolution in vision.
   - Accuracy before AlexNet (~70%) vs after (~85%+). GPUs made training feasible.

4. VGG (2014) - Visual Geometry Group
   - **Architecture:** Very deep (16–19 layers).
   - **Design Choice:** Small 3×3 filters stacked together.
     - Key Contribution:x
     - Showed that depth improves accuracy.
     - Standardized design → widely used as a baseline in research.

5. Key Takeaways
   - **LeNet:** Proof of concept.
   - **AlexNet:** Breakthrough in scale and performance.
   - **VGG:** Depth and simplicity as design principles.
   - Together, they form the **timeline of CNN evolution**.

ReLU and **Dropout**, two fundamental techniques in deep learning:

ReLU (Rectified Linear Unit)
   - **Definition:** An activation function used in neural networks.
   - **Meaning:** If the input is positive, keep it; if negative, output zero.
   - Why it matters:

    - Introduces **non-linearity** so networks can learn complex patterns.
    - Faster and simpler than older functions like sigmoid or tanh.
    - Helps reduce the vanishing gradient problem.
- Eg: Negative values are “cut off,” positives pass through.

Dropout

- **Definition:** A regularization technique to prevent overfitting.
- **How it works:** Randomly “drops” (sets to zero) a fraction of neurons during training.
- Why it matters:
    - Forces the network to learn **robust features** instead of memorizing.
    - Reduces reliance on specific neurons.
    - Improves generalization to unseen data.
- **Example:** Suppose a layer has 10 neurons. With dropout rate = 0.3, about 3 neurons are randomly ignored during each training pass. → The network learns to distribute knowledge across multiple neurons.

Quick Analogy

- **ReLU:** Like a filter that only lets positive signals through.
- **Dropout:** Like a team exercise where some members sit out randomly, so everyone learns to contribute — preventing over-reliance on a few.

Together, ReLU and Dropout made CNNs like **AlexNet** train faster and generalize better, which is why they were revolutionary in 2012.

Why Study Modern CNNs?

1. Introduction

- Deep CNNs evolved to address challenges of **depth, efficiency, and scalability**.
- Earlier models (LeNet, AlexNet, VGG) showed the power of CNNs, but deeper networks faced problems like vanishing gradients and computational inefficiency.
- Modern architectures like **ResNet** and **EfficientNet** solved these issues and set new benchmarks.

2. ResNet (2015)

- **Problem Solved:** Training very deep networks often led to vanishing gradients (weights stopped updating).
- Innovation: Introduced Residual Connections (skip connections).
    - Instead of learning direct mappings, the network learns *residuals* (differences).
    - Skip connections allow gradients to flow more easily through layers.
- **Impact:** Enabled training of networks with 50, 101, even 152 layers.
- **Achievement:** State-of-the-art performance on ImageNet.
- **Key Idea:** Learn residuals instead of direct mappings.

3. EfficientNet (2019)

- **Problem Solved:** Scaling networks (making them deeper, wider, or higher resolution) was inefficient and unbalanced.
- Innovation: Introduced compound scaling.

    - Scales depth, width, and resolution systematically using a single formula.
    - Achieves high accuracy with fewer parameters.
- **Impact:** EfficientNet models outperform larger CNNs while being lightweight.
- **Key Idea:** Balanced scaling across dimensions.

4. Key Takeaways
- **ResNet:** Made very deep networks trainable.
- **EfficientNet:** Made scaling efficient and practical.
- Together, they represent the **modern era of CNNs** — solving depth and efficiency challenges.
- These ideas influenced later models and even Vision Transformers.

Real-world applications: ResNet in medical imaging, EfficientNet in mobile vision apps.

Object Detection Architectures

1. Introduction
- Object detection = identifying *what* an object is and *where* it is in an image.
- Different architectures evolved to balance **accuracy vs speed**.
- We'll study three landmark models: **R-CNN, YOLO, SSD**.

2. R-CNN (2014)
- **Full name:** Region-based Convolutional Neural Network.
- Process:
    1. Generate region proposals (possible object locations).
    2. Run CNN on each region.
    3. Classify and refine bounding boxes.
- **Strengths:** High accuracy.
- **Weaknesses:** Very slow — thousands of regions per image.
- **Key Contribution:** First major CNN-based detector, proved deep learning works for detection.
- **Example:** How R-CNN takes an image of a dog, proposes multiple regions, and classifies each.

3. YOLO (2016)
- **Full name:** You Only Look Once.
- Process:
    - Treats detection as a single regression problem.
    - Predicts bounding boxes and class probabilities in one forward pass.
- **Strengths:** Real-time performance (fast).
- **Weaknesses:** Less accurate for small objects.
- **Key Contribution:** Made detection practical for real-time applications (e.g., self-driving cars).
- **Example:** How YOLO detects multiple objects in a street scene in one shot.

4. SSD (2016)
- **Full name:** Single Shot MultiBox Detector.
- Process:
    - Uses multiple feature maps at different scales.

    - Detects small and large objects simultaneously.
- **Strengths:** Faster than R-CNN, competitive accuracy.
- **Weaknesses:** Slightly less accurate than R-CNN for very complex scenes.
- **Key Contribution:** Balanced speed and accuracy, widely used in mobile vision tasks.
- **Example:** How SSD detects both a small cat and a large cat in the same image.

5. Key Takeaways
- **R-CNN:** Accurate but slow → research breakthrough.
- **YOLO:** Fast, real-time → practical applications.
- **SSD:** Balanced → efficient detection across scales.
- Together, they illustrate the trade-off between accuracy and speed in object detection.

- Which architecture would you use for medical imaging vs autonomous driving?
  For Medical Imaging → R-CNN (and its successors like Faster R-CNN)
- Why:
    - Medical imaging tasks (tumor detection, lesion identification) demand **high accuracy** and careful localization.
    - False negatives (missing a tumor) are unacceptable.
    - R-CNN's region proposal approach is slower but **precise**, making it suitable for offline analysis where speed is less critical.
- **Example:** Detecting small tumors in MRI scans — accuracy is more important than real-time speed.
- For Autonomous Driving → YOLO (or SSD)
- Why:
    - Self-driving cars need **real-time detection** of pedestrians, vehicles, and traffic signs.
    - YOLO processes the entire image in one pass, achieving high speed.
    - SSD also balances speed and accuracy, especially for detecting objects at multiple scales.
- **Example:** Detecting a pedestrian crossing the street — speed is critical to avoid accidents, even if accuracy is slightly lower than R-CNN.
- Key Insight
    - **Medical Imaging:** Accuracy > Speed → R-CNN.
    - **Autonomous Driving:** Speed > Accuracy → YOLO/SSD.
- The choice depends on the application's tolerance for errors vs need for real-time performance

Semantic Segmentation Architectures

1. Introduction
   - Semantic segmentation = assigning a **class label to every pixel** in an image.
   - Applications: medical imaging, autonomous driving, satellite imagery.
   - Challenge: Need both **context** (what the object is) and **precision** (where it is).
   - Two landmark architectures: **FCN** and **U-Net**.

2. Fully Convolutional Networks (FCN)
   - **Problem Solved:** Traditional CNNs ended with fully connected layers → lost spatial information.
   - **Innovation:** Replace fully connected layers with **convolutional layers**.
   - **Output:** Dense pixel-wise prediction map.
   - **Key Idea:** End-to-end segmentation using convolutional layers only.
   - **Strengths:** Simple, effective, first major CNN for segmentation.
   - **Weaknesses:** Coarse predictions, struggles with fine boundaries.
   - **Example:** How FCN takes an image of a cat and outputs a mask labeling every pixel as “cat” or “background.”

3. U-Net (2015)
   - **Architecture:** Encoder-decoder with **skip connections**.
     - **Encoder (downsampling path):** Captures context by reducing resolution.
     - **Decoder (upsampling path):** Recovers spatial resolution.
     - **Skip connections:** Combine low-level (fine detail) and high-level (context) features.
   - **Strengths:** Precise segmentation, especially for small structures.
   - Applications: Widely used in medical image segmentation (tumors, cells, organs).
   - **Key Idea:** Combine low-level and high-level features for accurate pixel classification.
   - **Example:** how U-Net can segment tumors in MRI scans with sharp boundaries.

4. Key Takeaways
   - **FCN:** First step → proved CNNs can do segmentation.
   - **U-Net:** Improved precision → became the standard in medical imaging.
   - Both highlight the importance of **architecture design** in solving pixel-level tasks.

   - Why do you think skip connections help in U-Net?

Skip connections (also called *residual connections*) are links that **bypass one or more layers** in a neural network and feed the input directly into a later layer.
Instead of only passing information through every intermediate layer, the network also “skips” ahead, combining earlier features with later ones.

- Preserve fine details:
- In U-Net, the encoder compresses the image (downsampling), which can lose spatial detail. Skip connections bring back those fine details from early layers into the decoder.
- Combine low-level and high-level features:
- Early layers capture edges, textures, and shapes. Later layers capture context (what the object is). Skip connections merge both, giving precise segmentation.
- Improve gradient flow:

- They help gradients move backward more easily during training, reducing the risk of vanishing gradients.

Example
Imagine segmenting a medical image:
- Encoder learns *"this region looks like tissue"* but loses exact boundary details.
- Skip connections bring back the original edge information from early layers.
- Decoder combines both → sharp, accurate tumor boundaries.

Encoder
- **Role:** The encoder is the **downsampling path** of the network.
- What it does:
  - Takes the input image and gradually reduces its spatial resolution (height × width).
  - Uses convolution + pooling layers to capture **context** and **high-level features** (e.g., "this region looks like a tumor" or "this area is a car").
- **Analogy:** Like compressing a photo into a summary — you lose fine detail but keep the big picture.

Decoder
- **Role:** The decoder is the **upsampling path** of the network.
- What it does:
  - Gradually increases spatial resolution back to the original image size.
  - Uses transposed convolutions or upsampling layers to reconstruct **pixel-level predictions**.
  - Combines context from the encoder with fine details (via skip connections).
- **Analogy:** Like expanding the compressed photo back into full resolution — but now with labels for each pixel.

Why They Work Together
- **Encoder:** Learns *what* is in the image (semantic meaning).
- **Decoder:** Learns *where* it is (precise location).
- Together, they allow the network to produce **dense segmentation maps** where every pixel is classified.

Example: U-Net in Medical Imaging
- **Encoder:** Identifies that a region looks like a tumor.
- **Decoder:** Uses skip connections to recover exact boundaries of the tumor.
- Result: A segmentation mask that highlights the tumor pixel-by-pixel.

In short: **Encoder compresses for context, decoder reconstructs for precision.** Skip connections bridge the two, ensuring no important details are lost.

Fully Connected (Dense) Layers

- **Structure:** Every neuron connects to *all* neurons in the previous layer.
- **Input type:** Flattened vectors (no spatial structure preserved).
- **Purpose:** Learn global patterns and combine features for final classification.
- **Advantages:** Strong at capturing complex relationships across all features.
- **Limitations:**
    - Large number of parameters → computationally heavy.
    - Ignores spatial locality (e.g., pixel positions in images).

Convolutional Layers

- **Structure:** Neurons connect only to a *local receptive field* (small region of input).
- **Input type:** Preserves spatial structure (height × width × channels).
- **Purpose:** Extract local features like edges, textures, shapes.
- **Advantages:**
    - Fewer parameters → more efficient.
    - Captures spatial hierarchies (edges → patterns → objects).
    - Translation invariance (features detected regardless of position).
- **Limitations:**
    - Alone, they don't combine all features globally — usually followed by pooling and dense layers for final classification.

Key Difference

- **Fully Connected:** Treats input as a flat vector, learns global relationships.
- **Convolutional:** Preserves spatial structure, learns local features progressively.

In practice, CNNs use **convolutional layers for feature extraction** and **fully connected layers for decision-making** at the end.

Mask R-CNN

1. Introduction
   - Object detection traditionally outputs **bounding boxes + class labels**.
   - But many applications need **pixel-level precision** (instance segmentation).
   - **Mask R-CNN (2017)** extended Faster R-CNN to achieve this.

2. Architecture Overview
   - **Base:** Built on Faster R-CNN (which detects objects with bounding boxes).
   - **Extension:** Adds a **segmentation mask branch** for each Region of Interest (RoI).
   - Outputs:
     - Bounding boxes (where objects are).
     - Class labels (what objects are).
     - Pixel-level masks (precise shape of each object).
   - **Key Idea:** Detect objects and generate masks simultaneously.

3. How It Works
   1. **Region Proposal Network (RPN):** Suggests candidate object regions.
   2. **RoI Align:** Improves accuracy by aligning features precisely (fixes quantization issues from RoI Pooling).
   3. **Classification & Bounding Box Regression:** Identifies object type and refines location.
   4. **Mask Branch:** Predicts a binary mask for each object, giving pixel-level segmentation.

4. Applications
   - **Medical Imaging:** Tumour segmentation, organ boundaries.
   - **Autonomous Driving:** Detecting pedestrians, vehicles, and lane markings with precise shapes.
   - **Crowd Analysis:** Separating overlapping people in dense scenes.
   - **General Vision Tasks:** Any scenario needing both detection and segmentation.

5. Key Contributions
   - Extended detection to **instance segmentation**.
   - Introduced **RoI Align** for better accuracy.
   - Became a benchmark model for segmentation tasks.

- **Why do you think pixel-level masks are more useful than bounding boxes in medical imaging?**

Pixel-level masks are more useful than bounding boxes in **medical imaging** because they provide **fine-grained detail** that bounding boxes cannot capture. Here's why:

1. Precise Localization
   - Bounding boxes only give a rough rectangular outline.
   - Tumors, lesions, or organs often have irregular shapes.
   - Pixel-level masks trace the **exact boundaries**, which is critical for diagnosis and treatment planning.

2. Quantitative Analysis
   - Masks allow measurement of **size, area, and volume** of medical structures.

- Bounding boxes overestimate these values since they include background pixels.
- Example: Calculating tumor volume for chemotherapy dosage requires accurate segmentation.

3. Clinical Decision Support
   - Doctors need to know not just *where* something is, but *how much* and *how it spreads*.
   - Masks provide detailed maps for surgical planning, radiation therapy, or monitoring disease progression.

4. Handling Overlaps
   - In medical scans, structures often overlap (e.g., tumors within organs).
   - Bounding boxes can't separate overlapping regions.
   - Masks distinguish each structure at the pixel level.

Key Insight
- **Bounding boxes:** Good for general detection (e.g., "there's a tumor here").
- Pixel-level masks: Essential for medical imaging where accuracy, measurement, and shape analysis directly affect patient care.

In short: Pixel-level masks give the precision needed for medical diagnosis and treatment, while bounding boxes are too coarse for such sensitive tasks.

Sequence Modeling for Video

Sequence modeling is the process of building models that can understand and predict data that comes in a **sequence** (ordered steps over time).
Unlike static data (like a single image), sequences have **temporal dependencies i.e.** the current element depends on what came before.

1. Introduction
   - Video data = **sequence of frames** (images over time).
   - Challenge: Need to capture both **spatial features** (what is in each frame) and **temporal dependencies** (how things change across frames).
   - Applications: action recognition, gesture recognition, video captioning, surveillance, autonomous driving.

2. CNNs for Spatial Features
   - **Role:** Extract features from individual frames.
   - **What they capture:** Objects, shapes, textures, edges.
   - **Limitation:** CNNs alone don't capture motion or order between frames.
   - **Example:** A CNN can recognize a hand shape in one frame, but not the gesture across multiple frames.

3. RNNs / LSTMs for Temporal Dependencies
   - **Role:** Model sequences by remembering past information.
   - **Why needed:** Video = temporal data → order and motion matter.
   - LSTMs (Long Short-Term Memory networks):
     - Handle **long-range dependencies** better than vanilla RNNs.

    - Use **gates**:
        - **Input Gate:** Decides what new info to store.
        - **Forget Gate:** Decides what past info to discard.
        - **Output Gate:** Decides what info to pass forward.
    - Useful for variable-length sequences (videos can be short or long).
- **Example:** Recognizing a "wave" gesture requires remembering motion across several frames.

4. CNN + LSTM Pipeline
- **Step 1:** CNN extracts spatial features from each frame.
- **Step 2:** LSTM processes the sequence of feature vectors across frames.
- **Step 3:** Output layer predicts the action/gesture class.
- **Result:** End-to-end system that understands both *what* is in the frame and *how* it changes over time.

5. Applications
- **Action Recognition:** Sports analysis, surveillance.
- **Gesture Recognition:** Sign language interpretation, human-computer interaction.
- **Video Captioning:** Generating textual descriptions of video content.
- **Autonomous Driving:** Understanding motion of pedestrians and vehicles.

6. Key Takeaways
- CNNs → spatial features.
- LSTMs → temporal dependencies.
- Together → powerful sequence modeling for video tasks.
- This pipeline is foundational for modern video AI systems.

RNNs and **LSTMs** in the context of sequence modeling:

Recurrent Neural Networks (RNNs)
- **Purpose:** Designed for **sequential data** (text, speech, video frames, time series).
- How they work:
    - Process input step by step.
    - Maintain a **hidden state** that carries information forward across time steps.
- Strengths:
    - Capture temporal dependencies (order, context).
    - Useful for tasks like language modeling, speech recognition, video analysis.
- Limitations:
    - Struggle with **long sequences** due to the **vanishing gradient problem** (information fades as it moves through many steps).
    - Training can be slow because of sequential processing.

Long Short-Term Memory Networks (LSTMs)
- **Purpose:** A special type of RNN designed to handle **long-range dependencies**.
- **Innovation:** Introduced **gates** to control information flow:
    - **Input Gate:** Decides what new information to store.
    - **Forget Gate:** Decides what past information to discard.
    - **Output Gate:** Decides what information to pass forward.
- Strengths:

    - Can remember information over long sequences.
    - Handle variable-length input (e.g., videos of different durations).
- **Applications:** Action recognition, gesture recognition, machine translation, video captioning.

RNN vs LSTM

Aspect	RNN	LSTM
Memory	Short-term	Long-term (handles long dependencies)
Problem	Vanishing gradients	Solves vanishing gradients
Best For	Simple sequences	Complex, long sequences
Example	Predicting next word in a short sentence	Understanding context across a long paragraph or video

Key Insight

- **RNNs** are the foundation for sequence modeling.
- **LSTMs** extend RNNs with gating mechanisms, making them practical for real-world tasks where long-term context matters (like video or language).
- **RNN = short-term memory** (remembers recent steps).
- **LSTM = long-term memory** (remembers important information across many steps).

- What features would CNN capture? What sequence would LSTM capture?

CNN (Convolutional Neural Network)

- **Role:** Extracts **spatial features** from each frame of the video.
- What it captures:
    - **Objects:** e.g., a hand, a ball, a car.
    - **Shapes & edges:** outlines of gestures or body parts.
    - **Textures & patterns:** clothing texture, background details.
- **Analogy:** Think of CNN as a *photographer* who takes snapshots of each frame and highlights what's inside the picture.

LSTM (Long Short-Term Memory Network)

- Role: Models temporal dependencies across frames.
- What it captures:
    - **Motion:** how the hand moves from left to right.
    - **Order:** the sequence of frames (first → next → last).
    - **Dynamics:** speed, rhythm, and continuity of actions.
- **Analogy:** LSTM is like a *storyteller* who connects the snapshots into a narrative, remembering what happened before and predicting what comes next.

Combined Pipeline (CNN + LSTM)

- **CNN:** Extracts features per frame (what is in the image).
- **LSTM:** Connects those features across time (how they change).
- **Output:** Action or gesture class (e.g., "waving," "jumping," "clapping").

Example

- Gesture Recognition:
  - CNN: Detects hand shape in each frame.
  - LSTM: Recognizes the sequence of movements → “wave.”
- Action Recognition:
  - CNN: Identifies a person and a ball.
  - LSTM: Sees the sequence → “throwing ball.”

In short: CNNs capture the *what* (spatial features), LSTMs capture the *when/how* (temporal sequence). Together, they make video understanding possible.

Features Captured by RNNs

1. Sequential Patterns
   - RNNs process data step-by-step, so they capture **order** in sequences.
   - Example: In a sentence, they learn that “the cat” comes before “sat on the mat.”

2. Temporal Dependencies
   - They remember **what happened before** to influence the current step.
   - Example: In video, they capture how a hand moves frame-to-frame to form a gesture.

3. Contextual Information
   - RNNs maintain a **hidden state** that carries context forward.
   - This helps them understand meaning based on prior inputs.
   - Example: In speech recognition, the word “bank” could mean riverbank or financial bank depending on earlier words.

4. Short-Term Memory
   - Standard RNNs are good at remembering **recent inputs**.
   - They struggle with very long sequences (vanishing gradient problem), which is why LSTMs and GRUs were introduced.
   - Example: Predicting the next note in a melody based on the last few notes.

Key Insight

- CNNs → spatial features (what is in each frame).
- RNNs → sequential features (how things change over time).
- Together, they allow models to understand both *content* and *context*.

In short: RNNs capture the flow, order, and dependencies in sequential data — the “storyline” across time.

Vision Transformers (ViT) & Attention

1. Introduction
   - Traditional CNNs excel at spatial feature extraction but struggle with **long-range dependencies**.
   - Transformers, originally designed for NLP, introduced the **attention mechanism** to capture relationships across sequences.
   - Vision Transformers (ViT) adapt this idea directly to images.

2. Attention Mechanism
   - **Purpose:** Computes relationships between elements in a sequence.
   - Key Components:
     - **Query (Q):** What we're looking for.
     - **Key (K):** What we have.
     - **Value (V):** The information to retrieve.
   - Process:
     - Compare Query with Keys → similarity scores.
     - Use scores to weight Values → output is a **weighted sum**.
   - Benefits:
     - Captures **long-range dependencies** (e.g., pixels far apart but related).
     - Flexible: learns which parts of the input are most relevant.
   - **Example:** In a sentence, attention lets the model connect "bank" with "river" or "money" depending on context.

3. Vision Transformer (ViT)
   - Step 1: Split Image into Patches
     - Example: 16×16 patches.
     - Each patch is flattened and embedded → treated like a "token" (similar to words in NLP).
   - Step 2: Transformer Encoder
     - Processes the sequence of patch embeddings using attention.
     - Learns relationships across patches (global context).
   - Step 3: Classification Token
     - Special token summarizes image features.
     - Used for final classification (e.g., "cat" vs "dog").
   - **Key Idea:** Apply transformer architecture directly to images, bypassing convolutions.

4. Benefits of ViT
   - Captures **global context** better than CNNs.
   - Scales well with large datasets.
   - Flexible: can be adapted for classification, detection, segmentation.
   - Inspired many hybrid models (CNN + Transformer).

5. Applications
   - Image classification (ImageNet benchmarks).
   - Medical imaging (capturing global patterns).
   - Video understanding (extended with temporal attention).
   - Multimodal tasks (vision + language).

6. Key Takeaways

- **Attention:** Learns relationships across elements.
- **ViT:** Treats image patches like tokens → transformer encoder processes them.
- **Impact:** Shifted computer vision from CNN-centric to transformer-centric approaches.

- **Why might splitting an image into patches help capture global context?**

Splitting an image into patches helps capture **global context** because it changes how the model "sees" the image:

1. Treats Image Like a Sequence
   - By dividing the image into patches (e.g., 16×16 pixels), each patch becomes a **token**, similar to words in a sentence.
   - This allows the transformer to apply the **attention mechanism** across patches, learning relationships between distant regions.

2. Captures Long-Range Dependencies
   - CNNs focus on local neighborhoods (small receptive fields).
   - Splitting into patches lets the model directly compare **far-apart regions** (e.g., top-left corner and bottom-right corner).
   - Example: In a photo of a zebra, stripes in different parts of the body are related — attention across patches captures this.

3. Reduces Complexity
   - Processing the whole image pixel-by-pixel would be computationally huge.
   - Patches reduce the sequence length, making transformer processing feasible.
   - Each patch summarizes local detail, while attention layers connect them globally.

4. Balances Local + Global Features
   - Each patch embedding contains **local information** (edges, textures).
   - Attention across patches builds **global understanding** (object shape, scene context).
   - Together, this gives a holistic view of the image.

Key Insight

Splitting into patches is what makes Vision Transformers possible:
- **Local detail** preserved in patches.
- **Global context** captured by attention across patches.
- This combination allows ViTs to rival CNNs in vision tasks, especially when trained on large datasets.

In short: Patches turn an image into a sequence, enabling transformers to connect distant regions and capture global context efficiently.